

Compiler Design — 10-Page Master Quick Revision

High-Yield Formula Sheet, Decision Trees & Top 10 BIT Mesra PYQ Solutions

⚡ 10-Page Master Quick Revision — Compiler Design (CS24301)

Page 1: 47-Topic Master Syllabus Progress Checklist (Modules I – V)

Page 3: Lexical Analysis: Phases of Compiler, Input Buffering & Regular Expressions

Page 5: Context-Free Grammars, Left Recursion Elimination & Left Factoring

Page 7: Bottom-Up LR Parsing Hierarchy: LR(0), SLR(1), CLR(1) & LALR(1)

Page 9: Intermediate Code: 3AC, Quadruples, Triples & Multi-Dimensional Array Offsets

Page 2: The Tier-S High-Yield Core Topic Ranking & Exam Strategy

Page 4: Direct DFA Construction: Nullable, Firstpos, Lastpos & Followpos Rules

Page 6: Top-Down LL(1) Parsing: FIRST & FOLLOW Inductive Math & Table Construction

Page 8: Semantic Analysis & SDD: S-Attributed vs. L-Attributed Definitions

Page 10: Code Generation & Optimization: Basic Blocks, CFG, DAG, Data-Flow & Peephole

CS24301 Compiler Design — All 47 Topics Verified & Mastered

Module	Topic #	Topic Name	Key Focus
M1: Lexical (7)	1	Introduction & Cousins (P-I-C-A-L)	Preprocessor, Compiler, Assembler, Linker, Loader
	2	Structure of Compiler (6 Phases)	Analysis front-end vs. Synthesis back-end
	3	The Lexical Analyzer	Token, Lexeme, Pattern & Error Recovery
	4	Input Buffering Strategies	Two-Buffer Scheme & Sentinel Optimizations
	5	Specification of Tokens	Regular Expressions & Algebraic Properties
	6	Recognition of Tokens	Transition Diagrams & Finite Automata
	7	Direct DFA Construction	Nullable, Firstpos, Lastpos, Followpos
M2: Syntax (10)	8	Introduction & CFG Grammars	Grammar Formal 4-tuple & Ambiguity Proofs
	9	Grammar Transformations	Left Recursion Elimination & Left Factoring
	10	Recursive Top-Down Parsers	Recursive Descent Parsing & Backtracking
	11	Non-Recursive Top-Down	Stack-driven LL(1) Parsing Table Model
	12	LL(1) Parser Design	FIRST & FOLLOW Math + Table Construction
	13	Bottom-Up Shift-Reduce	Handles, Handle Pruning & Viable Prefixes
	14	Variants of LR Parsers	LR(0), SLR(1), CLR(1), LALR(1) Hierarchy
	15	Parsing Conflict Handling	Shift-Reduce & Reduce-Reduce Disambiguation
	16	Syntax Error Detection	Panic-Mode & Phrase-Level Recovery
	17	Syntax Error Reporting	Error Productions & Diagnostic Compilers
M3: Semantics (9)	18	Semantic Analysis Overview	Static vs. Dynamic Semantic Checks
	19	Syntax-Directed Definitions	SDD Attribute Grammars & Dependency Graphs
	20	SDTS Translation Schemes	Embedded Semantic Actions in Productions
	21	SDTS for Declarations	Type Propagation & Scoped Symbol Tables
	22	Three-Address Code (TAC)	Quadruples, Triples & Indirect Triples
	23	Types of Attributes	S-Attributed vs. L-Attributed Definitions
	24	Type Checking	Name vs. Structural Equivalence & Coercion
	25	ICG for Assignments	Scalar Arithmetic & Temporary Allocation
	26	Multi-D Array Translation	Row/Col Major Linear Offset Derivations
M4: Runtime (9)	27	Boolean Expression ICG	Short-Circuit Boolean Evaluation Semantics
	28	Control Flow Translation	`if-else`, `while`, `for`, `switch-case` Jumps
	29	Backpatching Mechanics	`makelist`, `merge`, `backpatch` Operations
	30	Forward & Backward Jumps	Jump Labels & Control Flow Synthesis
	31	Procedure Calls & Returns	`param`, `call`, `return` & Parameter Modes
	32	Runtime Storage Layout	Code, Static, Heap & Call Stack Organization

Module	Topic #	Topic Name	Key Focus
	33	Activation Records (Stack)	7 P-R-C-A-S-L-T Frame Fields & Memory Map
	34	Storage Allocation Rules	Static vs. Stack vs. Heap Allocation
	35	Non-Local Data Access	Static Links Chain vs. Displays Array
M5: Opt & CG (12)	36	Code Generator Design	Instruction Selection & Target Architecture
	37	Target Language Costs	Instruction Costs & Target Addressing Modes
	38	Addresses in Target Code	Static Offsets & Stack Frame Relocation
	39	Basic Block Construction	The 3 Classical Leader Identification Rules
	40	Control Flow Graphs (CFG)	Basic Block Graph, Dominators & Loops
	41	Next-Use & Liveness	Variable Liveness inside Basic Blocks
	42	Register Allocation	Chaitin-Briggs Graph Coloring Algorithm
	43	Directed Acyclic Graphs	DAG Construction for Local Block Optimization
	44	Data-Flow Frameworks	Reaching Definitions, Available Expressions
	45	Loop Optimizations	Loop Invariant Code Motion & Strength Red.
	46	Peephole Optimization	Sliding Window Redundant Instruction Removal
	47	Simple Code Generation	Register & Address Descriptors ('getReg')

Page 2: The Tier-S High-Yield Core Topic Ranking & Exam Strategy

Rank	Core Exam Topic	Module	Frequency & Yield	Essential Mathematical Formula / Rule
★ 1	Direct DFA Construction	M1	100% Exam Probability (10M)	$\text{followpos}(i)$ on Syntax Tree Cat/Star nodes; $\text{DTran}[S, a] = \bigcup_{p \in S} \text{followpos}(p)$
★ 2	Left Recursion Elimination	M2	100% Exam Probability (5M)	$A \rightarrow A\alpha \mid \beta \implies A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$
★ 3	LL(1) Parser Construction	M2	100% Exam Probability (10M)	$\text{FIRST}(\alpha)$ and $\text{FOLLOW}(A)$ calculation tables + Parsing Table $M[A, a]$
★ 4	LR Parsing Power Hierarchy	M2	100% Exam Probability (8M)	$\text{LR}(0) \subset \text{SLR}(1) \subset \text{LALR}(1) \subset \text{CLR}(1)$; LALR(1) merges core states
★ 5	S- vs. L-Attributed SDD	M3	100% Exam Probability (8M)	Synthesized attributes (Bottom-Up LR) vs. Inherited left-to-right attributes (Top-Down LL)
★ 6	Multi-Dimensional Array Offsets	M3	100% Exam Probability (10M)	Row-Major: $\text{Address} = \text{base} + [(i_1 - l_1) \cdot n_2 + (i_2 - l_2)] \cdot w$
★ 7	Backpatching Control Flow	M4	100% Exam Probability (10M)	$\text{makelist}(i)$, $\text{merge}(p_1, p_2)$, $\text{backpatch}(p, \text{label})$ for boolean expressions
★ 8	Activation Record (Stack Frame)	M4	100% Exam Probability (8M)	The 7 Fields: Parameters, Return value, Control link, Access link, Saved status, Local data, Temporaries
★ 9	Basic Block Leader Rules	M5	100% Exam Probability (6M)	1. First instruction; 2. Target of branch; 3. Instruction immediately following branch
★ 10	Data-Flow Reaching Definitions	M5	100% Exam Probability (10M)	$\text{IN}[B] = \bigcup_{P \in \text{Pred}} \text{OUT}[P]$, $\text{OUT}[B] = \text{GEN}[B] \cup (\text{IN}[B] - \text{KILL}[B])$

Master Strategic Examination Execution Protocol

- Section A (2 Marks):** Target definitions: Token vs Lexeme, Compiler vs Interpreter, S- vs L-attributed, Static vs Dynamic scoping.
- Section B (5 Marks):** Short derivations: Left recursion elimination, Left factoring, Intermediate code triples/quadruples, Peephole rules.
- Section C (10–14 Marks):** Full numericals: Direct DFA from RE, LL(1) Table & Stack simulation, LR parsing DFA item sets, 3D Array memory calculations, Reaching Definitions data-flow convergence!

The 6 Compiler Phases & Memory Flow:

1. **Lexical Analysis:** Characters → Tokens. (DFA / Input Buffering).
2. **Syntax Analysis:** Tokens → Parse Tree / AST. (CFG / LL & LR Parsers).
3. **Semantic Analysis:** AST → Decorated AST. (Type Checking & SDD).
4. **Intermediate Code Gen:** Decorated AST → Three-Address Code (TAC).
5. **Code Optimization:** TAC → Optimized TAC (CFG, Data-Flow & Loops).
6. **Target Code Gen:** Optimized TAC → Target Assembly / Binary.

Concept	Formal Definition	Example
Token	Abstract syntactic terminal symbol category.	`IDENTIFIER`, `NUMBER`, `KW_IF`
Lexeme	Concrete source code character sequence matching pattern.	`"total_count"`, `"120"`, `"if"`
Pattern	Formal regular expression grammar rule for lexeme.	`[a-zA-Z_][a-zA-Z0-9_]*`

Input Buffering & Sentinel Optimization

- Naive buffering checks for buffer end AND current character (2 checks/char).
- Placing `eof` Sentinels at the end of each 4096-byte half-buffer reduces this to **1 check/char** in the common case!

The 4 Positional Syntax-Tree Functions:

- **nullable(**n**)**: Subexpression at node n can generate ϵ .
- **firstpos(**n**)**: Leaf positions matching first character of strings in $L(n)$.
- **lastpos(**n**)**: Leaf positions matching last character of strings in $L(n)$.
- **followpos(**i**)**: Positions j that can immediately follow position i .

Node n	$\text{nullable}(n)$	$\text{firstpos}(n)$	$\text{lastpos}(n)$
Leaf ϵ	true	$\emptyset$	$\emptyset$
Leaf pos i	false	$\{i\}$	$\{i\}$
$c_1 \mid c_2$	$\text{nullable}(c_1) \vee \text{nullable}(c_2)$	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$
$c_1 \cdot c_2$	$\text{nullable}(c_1) \wedge \text{nullable}(c_2)$	if $\text{nullable}(c_1)$ then $\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$ else $\text{firstpos}(c_2)$	if $\text{nullable}(c_2)$ then $\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$ else $\text{lastpos}(c_1)$
c_1^*	true	$\text{firstpos}(c_1)$	$\text{lastpos}(c_1)$

Hopcroft DFA Minimization Algorithm

Initialize $P = \{F, S - F\}$. Repeatedly split groups G on input $a \in \Sigma$ until no further partitions can be made. Merges all equivalent states in $O(n \log n)$ time!

Grammar Transformation Formulas:

1. **Immediate Left Recursion:** $A \rightarrow A\alpha \mid \beta \implies \mathbf{A} \rightarrow \beta\mathbf{A}', \quad \mathbf{A}' \rightarrow \alpha\mathbf{A}' \mid \epsilon$
2. **Left Factoring:** $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma \implies \mathbf{A} \rightarrow \alpha\mathbf{A}' \mid \gamma, \quad \mathbf{A}' \rightarrow \beta_1 \mid \beta_2$

Condition for LL(1) Grammar

A grammar G is LL(1) if and only if for every pair of productions $A \rightarrow \alpha \mid \beta$:

1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$
2. If $\alpha \Rightarrow^* \epsilon$, then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$.

FIRST and FOLLOW Rules Reference

- $\text{FIRST}(X)$: Terminals starting strings derived from X . Include ϵ if $X \Rightarrow^* \epsilon$.
- $\text{FOLLOW}(A)$: Terminals appearing immediately to right of A in some sentential form. Always include $\$$ in $\text{FOLLOW}(S)$.

Parser	Item Format	Reduction Rule	States	Power
LR(0)	$[A \rightarrow \alpha \cdot \beta]$	Placed in ALL terminal columns	K	Weakest
SLR(1)	$[A \rightarrow \alpha \cdot \beta]$	Placed ONLY in FOLLOW(A) columns	K	Moderate
CLR(1)	$[A \rightarrow \alpha \cdot \beta, a]$	Placed ONLY for exact lookahead a	$5 \times$ to $10 \times K$	Most Powerful
LALR(1)	Merged $[A \rightarrow \alpha \cdot \beta, a \mid b]$	Merges CLR(1) states with identical LR(0) cores	K	Industry Standard

Shift-Reduce vs Reduce-Reduce Conflicts

- **Shift-Reduce:** Parser cannot decide whether to shift token or reduce handle (resolve by favoring Shift).
- **Reduce-Reduce:** Parser cannot decide between two productions to reduce by (grammar flaw).

Page 7: Module III — Semantic Analysis, SDD & Type Checking

Dimension	S-Attributed Definitions	L-Attributed Definitions
Attributes Used	Synthesized attributes ONLY	Synthesized AND Inherited attributes
Information Flow	Strictly Bottom-Up (Children → Parent)	Top-Down & Left-to-Right between siblings
Evaluation Traversal	Post-Order Traversal	Pre-Order / Depth-First Traversal
Parsing Engine	Evaluated on-the-fly during Bottom-Up LR reductions	Natural match for Top-Down LL recursive descent

Type Systems & Structural Equivalence:

- **Name Equivalence:** Types are identical only if declared under identical names.
- **Structural Equivalence:** Types are identical if internal memory layouts, fields, and sizes match.

Multi-Dimensional Array Address Formulas:

- **1D Array:** $\text{Address}(A[i]) = \text{base} + (i - \text{low}) \cdot w$
- **2D Row-Major:** $\text{Address}(A[i, j]) = \text{base} + [(i - l_1) \cdot n_2 + (j - l_2)] \cdot w$
- **2D Column-Major:** $\text{Address}(A[i, j]) = \text{base} + [(j - l_2) \cdot n_1 + (i - l_1)] \cdot w$
- **3D Row-Major:** $\text{Address}(A[i, j, k]) = \text{base} + [((i - l_1) \cdot n_2 + (j - l_2)) \cdot n_3 + (k - l_3)] \cdot w$

TAC Representation	Record Structure	Key Advantage
Quadruples	`(op, arg1, arg2, result)`	Free reordering during optimization passes
Triples	`(op, arg1, arg2)`	Saves memory; no temporary variable names
Indirect Triples	Array of pointers $\rightarrow$ Triples	Optimal reordering without altering triple memory

The 3 Backpatching Functions:

1. **makelist(i)**: Creates a new list containing instruction index i .
2. **merge(p_1, p_2)**: Concatenates two jump target lists.
3. **backpatch(p, L)**: Inserts label L into all instructions indexed in list p .

The 7 Activation Record Fields (P-R-C-A-S-L-T)

Parameters → Return value → Control link → Access link → Saved machine status → Local data → Temporaries.

Non-Local Variable Access

- **Static Links**: Linked pointer traversal ($O(k)$ time for depth k).
- **Displays Array**: Global array `d[depth]` providing guaranteed **$O(1)$** instant access!

Basic Block Leader Rules:

1. The first instruction of the TAC sequence is a Leader.
2. Any instruction that is the target of a jump is a Leader.
3. Any instruction immediately following a jump is a Leader.

Data-Flow Equations for Reaching Definitions:

$$\text{IN}[\mathbf{B}] = \bigcup_{\mathbf{P} \in \text{Pred}[\mathbf{B}]} \text{OUT}[\mathbf{P}]$$
$$\text{OUT}[\mathbf{B}] = \text{GEN}[\mathbf{B}] \cup (\text{IN}[\mathbf{B}] - \text{KILL}[\mathbf{B}])$$

Optimization Pass	Mechanism	Before → After
Constant Folding	Compile-time arithmetic	<code>`x = 10 * 20` → <code>`x = 200`</code></code>
Constant Propagation	Propagating constant values	<code>`x = 10; y = x + 5;`</code> → <code>`y = 15;`</code>
Loop-Invariant Motion	Hoisting out of loops	Statements independent of loop variables moved to pre-header
Strength Reduction	Cheaper operations	Replacing multiplication $i * 4$ with pointer additions or shifts
Dead Code Elimination	Removing unused writes	Deleting computations whose values are never read